

Trabalho Prático

Reconhecimento de Dígitos Manuscritos (MNIST)

com uma Biblioteca de Matrizes em C

Trabalho individual | Defesa oral individual | Linguagem: C (padrão C99)

1. Apresentação

Uma rede neural treinada para reconhecer dígitos manuscritos não passa, no fundo, de uma sequência de *multiplicações de matrizes* intercaladas por uma função simples aplicada elemento a elemento. Se você sabe ler um arquivo, alocar memória e multiplicar matrizes, você sabe executar uma rede neural.

Neste trabalho você implementará, em C, uma pequena **biblioteca de operações matriciais** e, sobre ela, a *propagação para frente* (*feedforward*) de uma rede já treinada que classifica imagens 20×20 da base MNIST. Os pesos foram treinados previamente e são fornecidos; **não há treinamento neste trabalho**. O seu programa deve ler os pesos de um arquivo, ler imagens no formato PGM, calcular a saída da rede e indicar o dígito reconhecido. Ao final, você medirá a qualidade do classificador.

Habilidades avaliadas

Leitura de arquivos texto e binários; alocação dinâmica de memória; representação e manipulação de matrizes via ponteiros; organização do código em funções com responsabilidades separadas; tradução de uma especificação matemática em código; análise quantitativa de resultados; capacidade de explicar cada decisão de implementação.

2. A arquitetura da rede

A rede tem três camadas: entrada, intermediária (oculta) e saída. Como as imagens têm $20 \times 20 = 400$ pixels, a camada de entrada tem 400 unidades. A camada oculta tem 25 unidades e a de saída tem 10 (uma por dígito possível).

Dada uma imagem como vetor coluna $x \in \mathbb{R}^{400}$ (pixels normalizados em $[0, 1]$), a propagação para frente é:

$$\begin{aligned} a^{(1)} &= x, & z^{(2)} &= W^{(2)}a^{(1)} + b^{(2)}, & a^{(2)} &= g(z^{(2)}), \\ z^{(3)} &= W^{(3)}a^{(2)} + b^{(3)}, & a^{(3)} &= g(z^{(3)}), \end{aligned}$$

onde $g(z) = \frac{1}{1 + e^{-z}}$ é a função *sigmoide*, aplicada elemento a elemento. As dimensões são $W^{(2)}: 25 \times 400$, $b^{(2)}: 25 \times 1$, $W^{(3)}: 10 \times 25$, $b^{(3)}: 10 \times 1$.

A previsão é o índice do neurônio de maior ativação em $a^{(3)}$:

$$\hat{i} = \arg \max_{i \in \{0, \dots, 9\}} a_i^{(3)}.$$

Convenção dos rótulos – leia com atenção

Por herança da base original, o dígito **0** é representado pela **classe 10**, e os dígitos 1 a 9 pelas classes 1 a 9. Assim, se o índice (base 0) do maior neurônio é $\hat{i}$, a *classe* é $\hat{i} + 1 \in \{1, \dots, 10\}$ e o *dígito impresso* é $(\hat{i} + 1) \bmod 10 \in \{0, \dots, 9\}$. Os rótulos no arquivo `lista.txt` seguem o padrão 1 a 10.

3. Formatos dos arquivos

3.1 Imagens: PGM (Portable Gray Map), tipo P5

PGM P5 tem cabeçalho em texto e dados em bytes brutos, sem compressão:

```
P5
20 20
255
<400 bytes de dados: um byte por pixel, linha a linha>
```

Regras do *parser*: linhas iniciadas por # são comentários e devem ser ignoradas; o valor máximo é sempre 255; os dados começam logo após o \n que encerra a linha do valor máximo. Use `fread` para os pixels, não `fscanf`.

3.2 Pesos: arquivo texto pesos.txt

O arquivo descreve a rede camada a camada. Após eventuais comentários (#), vem o número de camadas com pesos; em seguida, para cada camada: uma linha **saídas** **entradas** com as dimensões de W , depois os **saídas**×**entradas** valores de W (em ordem por linha) e, por fim, os **saídas** valores do viés b .

Exemplo ilustrativo (rede de brinquedo $4 \rightarrow 3 \rightarrow 2$)

```
# numero de camadas
2
# camada 1: W (3x4) e b (3)
3 4
0.10 0.20 -0.30 0.05
-0.40 0.11 0.07 0.90
0.33 -0.12 0.50 -0.08
0.01 -0.02 0.03
# camada 2: W (2x3) e b (2)
2 3
0.7 -0.5 0.2
-0.1 0.4 0.6
0.05 -0.05
```

No trabalho real são duas camadas: $W^{(2)}: 25 \times 400$ e $W^{(3)}: 10 \times 25$.

3.3 Conjunto de avaliação: lista.txt

Uma linha por imagem, no formato <caminho.pgm> <rótulo>, com o rótulo em 1 a 10:

```
imagens/img_0000.pgm 10
imagens/img_0001.pgm 3
imagens/img_0002.pgm 7
```

4. Etapas do trabalho

Etapa 1 – Biblioteca de matrizes

(30 pontos)

O coração do trabalho. Represente uma matriz por um **struct** com as dimensões e um **único** vetor de **double** alocado dinamicamente (armazenamento por linha, *row-major*):

```
typedef struct {
    int linhas;
    int colunas;
```

```
double *dados; /* row-major: dados[i*colunas + j] */
} Matriz;
```

Implemente, no mínimo:

```
Matriz *matriz_criar(int linhas, int colunas); /* aloca (zerada) */
void matriz_liberar(Matriz *m); /* libera */
Matriz *matriz_multiplicar(const Matriz *A, const Matriz *B); /* C = A*B */
void matriz_somar(Matriz *A, const Matriz *B); /* A <- A + B (mesma dim) */
void matriz_sigmoid(Matriz *m); /* g() elemento a elemento */
int matriz_argmax(const Matriz *m); /* indice do maior (vetor coluna) */
```

Verificação obrigatória: construa uma rede de brinquedo (como a do exemplo) à mão, calcule a saída no papel e confirme que sua biblioteca reproduz exatamente os mesmos números.

Questões para a defesa:

- Por que armazenar a matriz como um vetor 1D *row-major* em vez de `double**`? Como você converte o par (i, j) no índice do vetor?
- Em `matriz_multiplicar`, qual verificação de dimensões é necessária? O que pode acontecer se ela for removida?
- Qual é a complexidade de `matriz_multiplicar` para $A: (l \times n)$ e $B: (n \times c)$?
- Por que usar `double` (e não `float`) no acumulador do produto?

Etapa 2 – Leitura dos pesos

(15 pontos)

Implemente a leitura de `pesos.txt` para uma estrutura que guarde, para cada camada, W e b . Trate comentários (`#`) no início do arquivo e leia primeiro as dimensões, depois os dados.

```
/* Le a rede do arquivo. Retorna ponteiro alocado no heap, ou NULL em erro.
 * O chamador é responsável por liberar. */
Rede *rede_ler(const char *caminho);
void rede_liberar(Rede *r);
```

Tratamento de erros: retorne `NULL` e imprima mensagem em `stderr` se o arquivo não existir, se as dimensões forem inválidas ou se a alocação falhar.

Questões para a defesa:

- Mostre o trecho que ignora comentários. O que acontece com um `#` no *meio* de uma linha de dados?
- Por que ler as dimensões *antes* dos valores de W ? O que mudaria se a ordem W e b fosse trocada no arquivo?
- O seu leitor funcionaria para uma rede com 4 camadas sem alterar o código? Por quê?

Etapa 3 – Leitura de imagens PGM e vetorização

(15 pontos)

Leia uma imagem PGM P5 e devolva-a como vetor coluna 400×1 de `double`, com os pixels **normalizados** em $[0, 1]$ (valor dividido por 255).

```
/* Le PGM P5 e devolve vetor coluna (largura*altura x 1) normalizado em [0,1]. */
Matriz *pgm_ler_vetor(const char *caminho);
```

Por que normalizar?

A rede foi treinada com pixels em $[0, 1]$. Se você esquecer de dividir por 255, as ativações saturam e a acurácia despenca. Documente, no relatório, a acurácia *com* e *sem* normalização.

Questões para a defesa:

- Por que `fread` (e não `fscanf`) para os bytes de pixel?
- Por que ler os bytes como `unsigned char` e não `char`? Para quais valores isso faz diferença?
- O que deveria acontecer se a imagem não for 20×20 ? Como isso interage com a dimensão de $W^{(2)}$?

Etapa 4 – Inferência (propagação para frente)

(20 pontos)

Usando sua biblioteca, implemente a previsão: parta de $a^{(1)} = x$ e, para cada camada, calcule $z = Wa + b$, aplique a sigmoide, e siga para a próxima. Devolva o índice do maior neurônio da última camada e converta-o em dígito.

```
int prever(const Rede *r, const Matriz *x); /* índice 0..9 do maior neurônio */
```

Verificação obrigatória: a acurácia no conjunto fornecido deve ficar em torno de **97,5%**. Um valor muito menor indica erro de normalização, de orientação das matrizes ou de mapeamento de classe.

Questões para a defesa:

- A sigmoide é monótona crescente. O que isso implica sobre aplicar (ou não) a sigmoide na *última* camada antes do `argmax`? O resultado muda?
- Por que o dígito 0 corresponde à classe 10? Mostre seu mapeamento índice $\rightarrow$ dígito.
- Dada uma rede de brinquedo e uma entrada pequena, preveja a saída *no papel* antes de rodar o programa.

Etapa 5 – Avaliação quantitativa e relatório

(20 pontos)

Programa principal. O programa deve aceitar dois modos de uso:

```
./mlp <pesos.txt> <imagem.pgm> # preve uma imagem
-> Dígito previsto: D

./mlp <pesos.txt> --avaliar <lista.txt> # acuracia no conjunto
-> Acuracia: XX.XX% (acertos/total)
```

Experimentos obrigatórios (todos no relatório):

1. Acurácia global no conjunto fornecido (compare com $\approx 97,5\%$).
2. **Matriz de confusão** 10×10 e acurácia por dígito. Quais dígitos a rede mais confunde entre si?
3. **Ablação da não linearidade:** remova a sigmoide das camadas *ocultas* (deixando a rede puramente linear) e meça a acurácia. Explique a queda.
4. **Imagens próprias:** desenhe ao menos **dois** dígitos seus, converta-os para PGM 20×20 e teste. A rede acerta? Discuta o que facilita ou atrapalha.

Como gerar uma imagem PGM 20×20 a partir de um desenho

```
# ImageMagick:
convert meu_5.png -colorspace Gray -resize 20x20! -depth 8 meu_5.pgm
# ou em Python (Pillow):
from PIL import Image
Image.open("meu_5.png").convert("L").resize((20,20)).save("meu_5.pgm")
```

Atenção ao fundo: na MNIST o traço é *claro* sobre fundo *escuro*.

Discussão (responda em prosa, ao menos um parágrafo por pergunta):

- Por que a sigmoide na última camada não altera a classe prevista, mas a sua remoção nas camadas ocultas altera (e muito)?
- A acurácia global esconde quais informações que a matriz de confusão revela?
- Onde está o custo computacional dominante da inferência? Como ele cresce com o tamanho da camada oculta?

Reflexão sobre o processo: descreva uma decisão de implementação que você tomou e que não estava explícita no enunciado, por que a tomou, e o que teria acontecido se tivesse feito diferente.

5. Organização e requisitos técnicos

Estrutura de arquivos:

```
mnist/
  mlp.c # programa principal e todas as funcoes
  Makefile # ou instrucoes de compilacao no relatorio
  relatorio.pdf # (ou .docx) com os experimentos
  dados/
    pesos.txt
    lista.txt
    imagens/ # imagens fornecidas (*.pgm)
    minhas/ # suas imagens proprias (*.pgm)
```

Requisitos:

- Compilar sem avisos com `gcc -Wall -Wextra -std=c99 mlp.c -o mlp -lm`.
- Toda memória alocada com `malloc/calloc` deve ser liberada com `free`. Verifique com Valgrind se disponível.
- Funções com no máximo 40 linhas; se passar, quebre em auxiliares.
- Código de saída 0 em sucesso e 1 em erro.
- Sem variáveis globais.

O que não é permitido

Bibliotecas externas de álgebra linear, imagem ou aprendizado (BLAS, OpenCV, libpng, GSL etc.); funções prontas de multiplicação de matrizes; código copiado sem compreensão. A defesa verificará isso.

Sobre o uso de LLMs e geração de código

O uso de LLMs e ferramentas de geração de código é permitido, desde que **todo** o código entregue seja compreendido e explicável. Na defesa, qualquer trecho pode ser questionado: “o que esta linha faz?”, “por que esse tipo aqui?”, “o que acontece se este `if` for removido?”. Código que o aluno não consegue explicar resulta em nota zero na etapa correspondente.

6. Avaliação

Critério	Pontos	Como é avaliado
Etapa 1 – Biblioteca de matrizes	30	Código + verificação + defesa
Etapa 2 – Leitura dos pesos	15	Código + defesa
Etapa 3 – Leitura de PGM	15	Código + defesa
Etapa 4 – Inferência	20	Código + acurácia + defesa
Etapa 5 – Avaliação e relatório	20	Relatório + programa principal
Total	100	

Critérios de desconto:

- Código que não compila: -50% na etapa correspondente.
- Vazamento de memória detectado: -10% por função com *leak*.
- Relatório sem a matriz de confusão: -10% .
- Ausência de imagens próprias: -10% .
- Funções com mais de 40 linhas sem justificativa: -5% por função.

Na defesa, o professor pode pedir ao aluno:

- Explicar qualquer trecho do código linha a linha.
- Modificar o código ao vivo para testar uma variante (ex.: remover uma sigmoide).
- Prever o resultado para uma entrada específica antes de executar.
- Explicar o que aconteceria se uma função fosse implementada de outra forma.

A defesa não tem duração fixa – termina quando houver evidência suficiente para avaliar a compreensão do aluno.